Interpretable-by-Design Language Models via Activation-Based Gradient Routing

Minor Project | Mid-Term Evaluation

Department of CSE, PEC

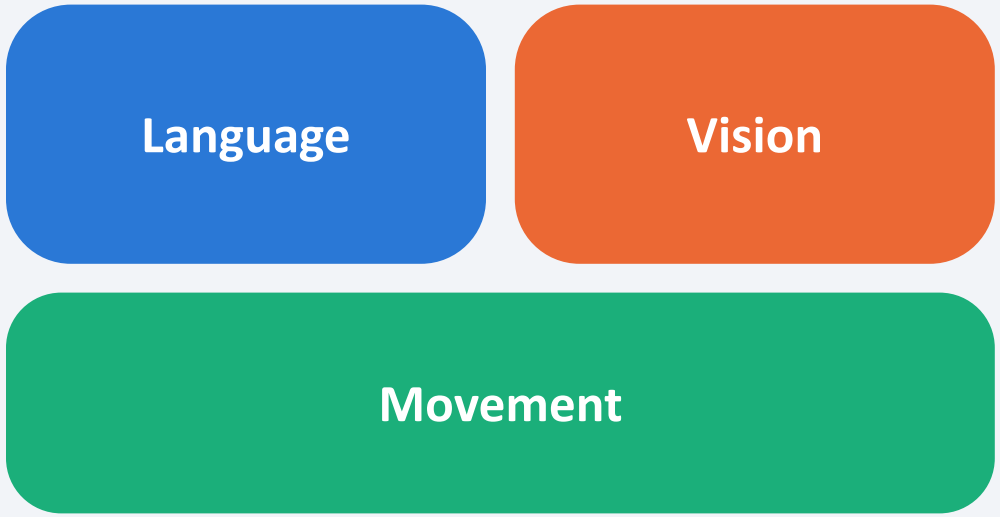
Team: [Name 1] · [Name 2] · [Name 3] · [Name 4]

Guide: [Guide name] | [Date]

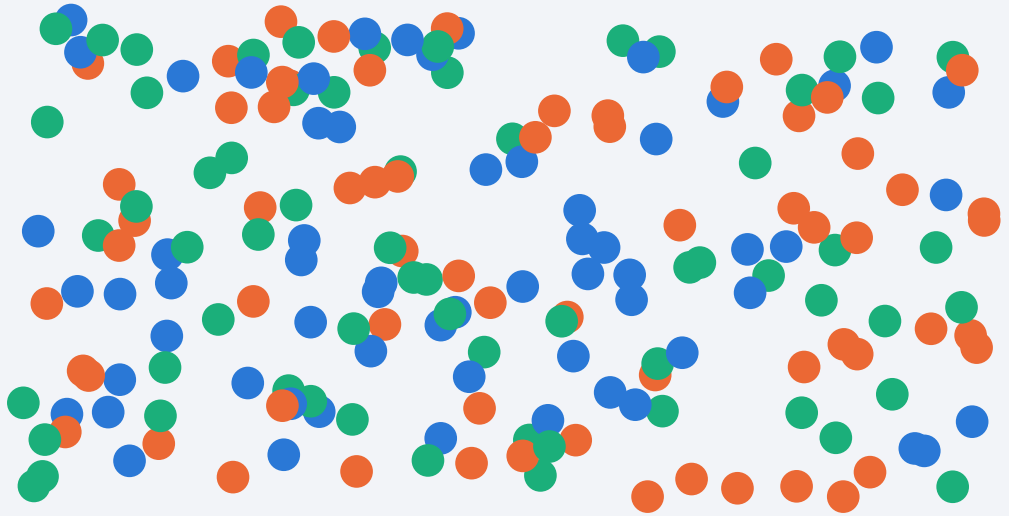
A language model is a black box

A language model **predicts the next word**. It learns by adjusting millions of numbers called **weights**.

A brain: specialised regions



A language model: no regions



Grammar, math and code are mixed across every weight.

Why the black box is a problem

1

Can't explain

We cannot point to where an answer came from.

2

Can't delete a skill

Unlearning: removing one skill, such as unsafe knowledge, without starting over.

3

Forgets old skills

Catastrophic forgetting: learning something new overwrites old knowledge.

Research question: can a model organise its own “brain regions” while it trains?

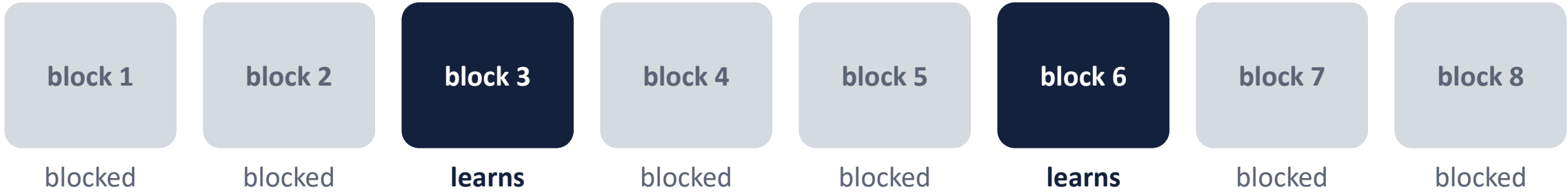
How our work differs from existing research

Approach	What decides the regions?	Goal	Needs human labels?
Mixture of Experts (Switch Transformer, Mixtral)	A learned router, word by word	Cheaper computing at large scale	No
Gradient Routing (Cloud et al., 2024)	Humans assign data to regions	Localise a skill, remove it later	Yes
EWC (Kirkpatrick et al., 2017)	An importance score for each weight	Don't forget old tasks	Task boundaries
Ours	**The model's own activity**	**Self-organised regions we can switch off**	**No**

The gap: Gradient Routing needs humans to assign data to regions. Ours lets the model decide.

Our idea: the most active blocks learn

Inside every layer, the **MLP** (the part that stores knowledge) is cut into **8 blocks**.



Each batch: only the 2 most active blocks get the learning signal (gradient). Which 2 varies.

1 Measure

How active is each block?

2 Select

Top 2 learn, 6 are blocked

3 Balance

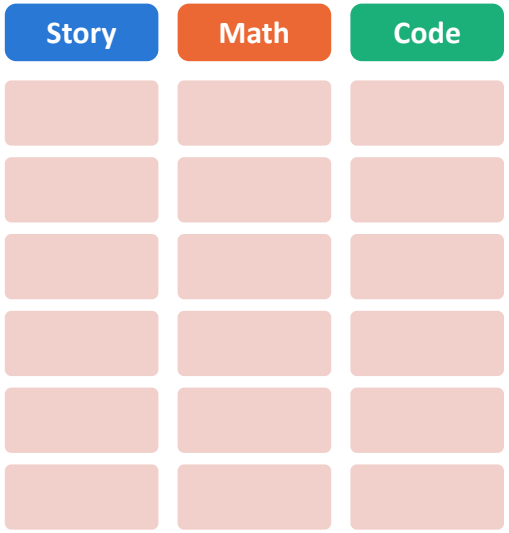
A nudge keeps idle blocks alive

Like brain regions: they specialise through use and competition.

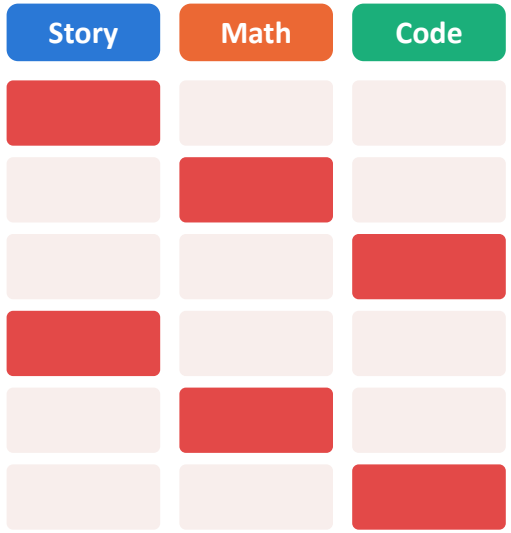
How we test it: switch one block off

Like studying brain injury: damage one region and see which ability is lost.

ILLUSTRATIVE SKETCH, NOT REAL DATA



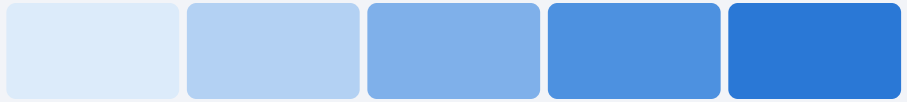
Dense model: every block matters a little for everything



Specialised model: each block matters for one skill

Each row: one block switched off.
Colour: how much the error rises for each skill.

Specialization score S



S = 0
skill shared by all

S = 1
one skill only

High S: switching off that block removes one skill.

What we built

10.6M

learned numbers
(29.9M with word table)

6 × 8 = 48

layers × blocks to study

19 min

to train on a free
Kaggle T4 GPU

Stories

TinyStories

20.0M tokens

"Once upon a time there lived a little girl…"

Math

Our own generator

10.0M tokens

Q: 347 + 589 = A: 936

Code

CodeParrot (Python)

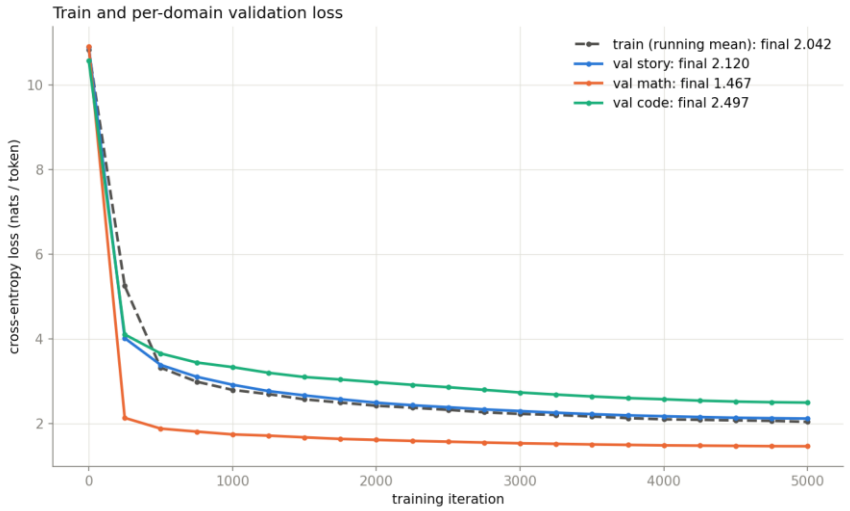
10.6M tokens

```
def add(a, b): …
```

Token = a word piece. 40.6M tokens in total.

Python | **PyTorch** | **nanoGPT-style GPT** | **Kaggle T4 GPU** | **GitHub** | **14 / 14 tests pass**

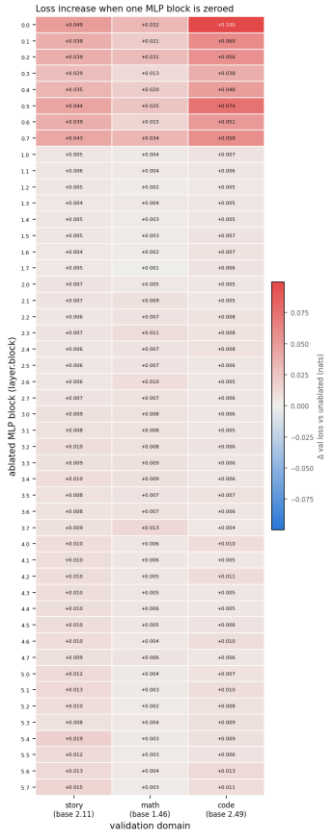
Results so far



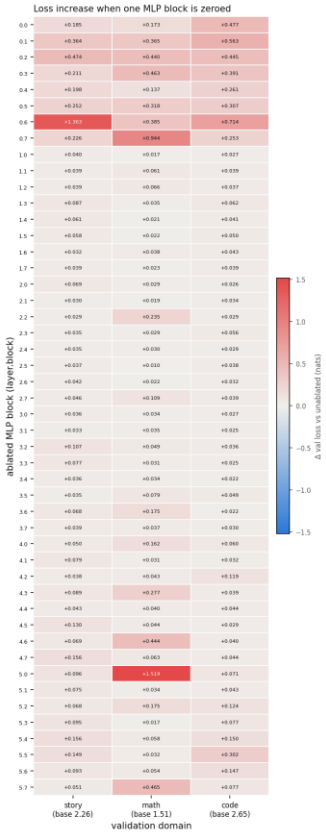
Loss curves (dense): lower is better

	Dense	Routed
Story loss	2.120	2.270
Math loss	1.467	1.510
Code loss	2.497	2.659
Mean S	**0.051**	**0.110**

Dense



Routed



Rows: blocks switched off. Darker = more damage. Scales differ.

Model output (dense)

Story

"Once upon a time there lived a tidy little girl. One day she went to the park with her mom."

Math

Q: 26 - 5 = A: 21

Code

def add(a, b): o = b.read() … return o
(looks like code, not valid yet)

Takeaway: routing doubled specialization (S 0.05 → 0.11) for a 3–7% loss cost. One run each, so not yet proven.

A key finding, and what is still open

Before: naive masking



9 of 16 blocks never selected (small test): the balancing signal was blocked too.

After: balancing signal not masked



No block left behind. Full run: all 48 blocks used.

Known limitations

Dominant block

Layer 0, block 6 took 70% of picks; most took 15–31%.

Mixed batches

A batch holds stories, math and code, so each pick averages them.

Optimizer drift

AdamW still nudges blocks we blocked.

Progress and next steps

Data pipeline
41M tokens, 3 domains
Completed

Baseline training
5,000 steps on a T4
Completed

Routing module
14 tests, 1 full run
Completed

Evaluation harness
Ablation done, probes pending
Ongoing

Next steps

1. Repeat with several seeds
2. Human-assigned regions baseline
3. Fix dominant block and drift
4. Consolidation phase
5. Domain probes
6. Unlearning and continual learning

Thank you · Questions?